



MetaBCI: An open-source platform for brain–computer interfaces[☆]

Jie Mei^{a,b,1}, Ruixin Luo^{a,b,1}, Lichao Xu^a, Wei Zhao^{a,b}, Shengfu Wen^a, Kun Wang^{a,c}, Xiaolin Xiao^{a,b,c}, Jiayuan Meng^{a,b,c}, Yongzhi Huang^{a,c}, Jiabei Tang^{d,c}, Longlong Cheng^e, Minpeng Xu^{a,b,c,*}, Dong Ming^{a,b,c}

^a Academy of Medical Engineering and Translational Medicine, Tianjin University, Tianjin, 300072, People's Republic of China

^b Department of Biomedical Engineering, College of Precision Instruments and Optoelectronics Engineering, Tianjin University, Tianjin, 300072, People's Republic of China

^c Haihe Laboratory of Brain–computer Interaction and Human–machine Integration, Tianjin, 300392, People's Republic of China

^d Tiankai Suishi (Tianjin) Intelligence Ltd., Tianjin, 300192, People's Republic of China

^e China Electronics Cloud Brain (Tianjin) Technology Co., Ltd., Tianjin, 300392, People's Republic of China

ARTICLE INFO

Keywords:

Brain–computer interface
Electroencephalogram
Open-source
One-stop platform
Python

ABSTRACT

Background: Recently, brain–computer interfaces (BCIs) have attracted worldwide attention for their great potential in clinical and real-life applications. To implement a complete BCI system, one must set up several links to translate the brain intent into computer commands. However, there is not an open-source software platform that can cover all links of the BCI chain.

Method: This study developed a one-stop open-source BCI software, namely MetaBCI, to facilitate the construction of a BCI system. MetaBCI is written in Python, and has the functions of stimulus presentation (*Brainstim*), data loading and processing (*Braindata*), and online information flow (*Brainflow*). This paper introduces the detailed information of MetaBCI and presents four typical application cases.

Results: The results showed that MetaBCI was an extensible and feature-rich software platform for BCI research and application, which could effectively encode, decode, and feedback brain activities.

Conclusions: MetaBCI can greatly lower the BCI's technical threshold for BCI beginners and can save time and cost to build up a practical BCI system. The source code is available at <https://github.com/TBC-TJU/MetaBCI>, expecting new contributions from the BCI community.

1. Introduction

The concept of brain–computer interface (BCI) was first proposed by Jacques J. Vidal in 1970 s [1]. Since then, the field of BCI has drawn increasing attention from researchers. BCIs translate brain activities into computer commands to replace, restore, enhance, supplement, and improve the central or peripheral nervous system [2–4]. In contrast to conventional human–computer interaction forms that are mainly dependent on hands, BCIs create a more direct interaction between the human brain and external devices. In the field of BCI research, non-invasive BCIs, which collect brain signals from the scalp, are favored in most cases because of their safety and convenience [5,6]. Among the non-invasive brain signal recording methods, electroencephalogram (EEG) is the most popular and has almost become a standard practice for BCIs. Currently, event-related desynchronization/synchronization

(ERD/ERS) [7–9], event-related potential (ERP) [10–12], and steady-state visual evoked potential (SSVEP) [13–16] are the three most prevalent EEG features used in non-invasive BCIs, which have been demonstrated to be capable of spelling characters [17], controlling devices like drones [18,19], robotic arms [20,21] and wheelchairs [22], etc.

A complete EEG-based BCI system usually consists of four main modules, including stimulus presentation, brain signal acquisition, brain signal processing, and command feedback [23]. These links with different functions require different considerations. The stimulus presentation module aims to encode computer commands into brain signals by guiding users to perform specific mental tasks. To get accurate EEG codes, the stimuli must be presented in precise timing. Currently, Psychtoolbox [24] (in Matlab), Psychopy (in Python) [25], and E-prime (in E-basic) have been widely applied in BCI studies to

[☆] Online demos can be found at: https://youtube.com/playlist?list=PL2KdW-WO2hd_ZgVDevX7t22P2V33BlonG&si=yM5sLXGoWp3RfdSA.

* Corresponding author at: Academy of Medical Engineering and Translational Medicine, Tianjin University, Tianjin, 300072, People's Republic of China. E-mail addresses: chmeijie@tju.edu.cn (J. Mei), ruixin_luo@tju.edu.cn (R. Luo), xmp52637@tju.edu.cn (M. Xu).

¹ Contributed equally to this work.

The List of abbreviations

Abbreviation	Full name
BCI	brain–computer interface
EEG	electroencephalogram
ERD	event-related desynchronization
ERS	event-related synchronization
ERP	event-related potential
SSVEP	steady-state visual evoked potential
LSL	lab streaming layer
MI	motor imagery
aVEP	asymmetric visual evoked potential
GEP	generalized eigenvalue decomposition
CSP	common spatial pattern
MultiCSP	multiclass CSP
FBCSP	filter bank CSP
CCA	canonical correlation analysis
TRCA	task-related component analysis
SSCOR	sum of squared correlation
TDCA	task discriminant component analysis
SKLDA	shrinkage linear discriminant analysis
STDA	spatial–temporal discriminant analysis
DSP	discriminative spatial pattern
DCPM	discriminative canonical pattern matching
SPD	symmetric positive-definite
MDRM	minimum distance to Riemannian mean
FGDA	fisher geodesic discriminant analysis
FgMDRM	fisher geodesic MDRM
TSClssifier	tangent space classifier
RA	Riemannian alignment
rRA	recursive RA
ttCCA	transfer template-based CCA
RPA	Riemannian procrustes analysis
LST	least-squares transformation
MEKT	manifold embedded knowledge transfer
ITR	information transfer rate
TPR	true positive rate
FNR	false negative rate
FPR	false positive rate
TNR	true negative rate
ROC	receiver operating characteristic curve
AUC	area under curve
FBTRCA	filter bank TRCA
FBTDCA	filter bank TDCA
FBeCCA	filter bank extended CCA
SVM	support vector machine

present stimuli. The signal acquisition module often collects brain signals from the scalp and converts them into digital signals for storage or online processing. Nowadays there are a number of EEG acquisition devices available for the construction of BCI systems, such as SynAmps RT (Neuroscan Inc), Neusen W (Neuracle Inc), EGI (Philips Inc), g. Nautilus (g.tec Inc), and ActiveTwo (Biosim, Inc), which results in a diversity of storage formats and transfer protocols of EEG data. Therefore, it is time-consuming to address their differences in loading and real-time fetching data. Some toolkits, such as the Lab streaming layer [26] (LSL, in C++) and BCI 2000 [27] (in C++), provide unified pathways to fit different devices in recording, loading, and retrieving EEG data. The signal processing module translates EEG signals into computer commands, and includes preprocessing, feature extraction, and pattern recognition.

Existing toolkits, such as EEGLAB [28] (in Matlab), MNE [29] (in Python or C language), FieldTrip [30] (in Matlab), and Brainstorm [31] (in Matlab), can meet part needs of signal processing for BCIs. They put more emphasis on EEG feature visualization and analysis but are weak in the recognition of EEG patterns. Compared with those toolkits, MOABB [32] orients towards building standard pipelines of EEG feature extraction and pattern recognition, and establishes a benchmark for the evaluation of EEG decoding algorithms. The command feedback module grants users access to the outputs generated by a BCI. For a natural interaction, decoded commands should be fed back to the user in real-time with the appropriate form, i.e., visual, tactile, or auditory, etc. To the best of our knowledge, there are no existing BCI toolkits that can cover all links of the BCI chain from stimulus presentation, data collection, and processing to command feedback. Therefore, it is not easy for researchers, especially for BCI beginners, to build up a BCI system, as different links in the BCI chain require developers to have different expertise.

To lower the technological barrier and research costs for BCI studies, we developed a novel open-source platform in Python for non-invasive BCI, namely MetaBCI. Python language makes MetaBCI rich in functions, easy to use, and flexible cross-platform. MetaBCI consists of *Brainstim*, *Brainda*, and *Brainflow*. *Brainstim* provides solutions for rapidly creating and presenting BCI experimental stimuli. *Brainda* offers a complete workflow for offline analysis, including unified methods for data loading, formatting, and decoding. For implementing online BCI systems, *Brainflow* is dedicated to real-time data acquisition, processing, and result feedback. These three parts of MetaBCI can cover all links of the BCI chain, which makes it easy for researchers to begin a BCI study and will inspire more ideas from the BCI community.

The remainder of this paper is structured as follows: Section 2 reviews the related literature; Section 3 describes the framework and main features of MetaBCI; Section 4 demonstrates the effectiveness of MetaBCI using six cases (3 offline experiments and 3 online experiments); and Section 5 compares the differences between MetaBCI and other BCI toolkits, and discusses the potential of MetaBCI. Finally, this paper is summarized in Section 6.

2. Related literature

Researchers developed software platforms to assist users in conveniently building BCI systems. Some psychology toolkits, including Psychtoolbox [24] in Matlab, Psychopy [25] in Python, and E-prime in E-basic, are initially used to design psychology experiment tasks, which have been also applied to present stimuli of BCI paradigms due to their high precision in timing. Some signal processing toolkits, primarily including EEGLAB [28] in Matlab, FieldTrip [30] in Matlab, Brainstorm [31] in Matlab, MNE [29] in Python, and MOABB in Python [32] can implement some basic or advanced data processing methods. They can also be easily used to analyze BCI data. These toolkits often work well for parts of BCI links and can be classified according to their functions. Despite providing convenience in BCI development, they usually require collaboration to build a complete BCI system.

To further lower the BCI's technical threshold, researchers have been dedicated to developing one-stop software platforms for BCI in recent years. Initially, the primary goal of BCI software development was to resolve the issue of the BCI field lacking a unified software design framework. In 2004, the Wadsworth Center in the USA introduced a general-purpose BCI system, BCI2000, which offered a unified design model for BCI [27]. The BCI2000 divided the system into four functional modules, which include: the source module, signal processing module, user application module, and operator module. The users needed to add code within different modules to build a BCI. As BCI technology continues to develop, its application areas have also broadened. The development of BCI software has shifted its focus towards the flexibility and compatibility of the platform. In 2010, the OpenViBE software platform, which enabled researchers to design and

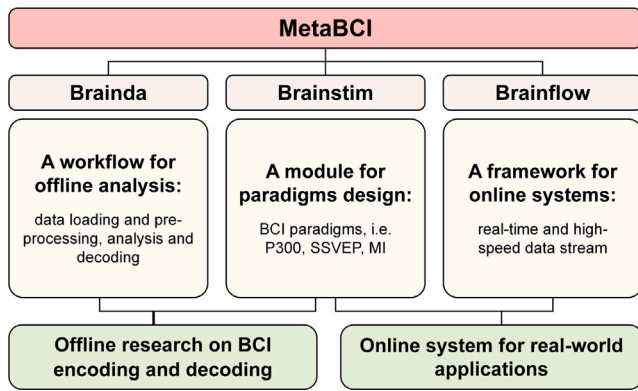


Fig. 1. Overview of MetaBCI.

use BCIs, was proposed to facilitate the application of BCIs in virtual reality (VR) [33]. OpenViBE had high modularity, and it embedded tools for visualization and feedback based on VR and 3D displays. The results showed its suitability for the design of VR applications controlled with a BCI. For a detailed comparison of BCI toolkits, please see Table 4 in the Discussion section.

3. Method

3.1. Overview of MetaBCI

MetaBCI is a one-stop software for the construction of BCIs, which consists of three parts: *Brainda*, *Brainstim*, and *Brainflow* (Fig. 1). For offline analysis, *Brainda* unifies the data format of different datasets, optimizes the pre-processing approach, and offers widely used decoding algorithms in BCIs. *Brainstim* empowers users to efficiently construct the user interface for a BCI by incorporating widely used paradigms while also providing flexibility for incorporating custom paradigms. *Brainflow* is a framework for developing online BCI systems, which can realize real-time high-speed data retrieving, data processing, and output feedback.

MetaBCI provides a user-friendly way to start BCI research and develop BCI applications. It is programmed with the internationally popular open-source language Python. Fig. 2 presents the example scripts on how to use MetaBCI. Currently, it comprises 383 classes (or functions), 53 brain-computer decoding methods, and provides access to 17 BCI public datasets.

3.2. Main features

3.2.1. Brainda : offline data loading and processing

Brainda is a workflow for BCI offline analysis, including data loading, pre-processing, and decoding. The pipeline of *Brainda* is presented in Fig. 3 and each step is detailed below.

Currently, there are various data formats for saving datasets in the BCI community. Different data loading processes are required when using different datasets, reducing the efficiency of BCI research. To this end, *Brainda* supports auto-downloading datasets from the web and reformatting them as *single_raw_data* or *multiple_raw_data*, which represent Raw data in MNE-Python for a single subject or multiple subjects respectively [29]. Then, *Brainda* provides three consecutive stages, i.e. Raw, Epoch, and Data stages. In all stages, users can add customized methods by using externally configurable *hook()* functions to meet different pre-processing needs.

After data loading and pre-processing, the function *get_data()* shall be used to obtain BCI data in a uniform format (X , y , and $Meta$). X represents the multi-channel EEG data of all trials and y represents the corresponding labels. $Meta$ is in the DataFrame structure. It gives extra

descriptions of experimental information e.g., subject, session, and run, for users to select data based on their needs. Fig. 2(a) and 2(b) show the usage examples of data loading and pre-processing in MetaBCI.

The advanced decoding methods in the BCI community are often implemented using different programming languages, workflows, and validation methods. It is difficult for BCI researchers to reproduce and compare these methods. To address this problem, *Brainda* standardizes the methods' usage interface following the commonly used machine learning toolkit Scikit-Learn [34]. It implements a general BCI decoding workflow, where function *fit()* estimates models using the training data, *transform()* extracts features of data, and *predict()* predicts the labels of the testing data. Notably, the pipeline mechanism enables the concatenation of *Brainda* and Scikit-Learn models. An example of data decoding is shown in Fig. 2(c).

3.2.2. Brainstim : experiment paradigm presentation

The BCI paradigms act as encoders that modulate the brain signal to generate specific features, e.g., P300 [35], SSVEP [14], motor imagery (MI) [36], and asymmetric visual evoked potential (aVEP). All of them require a stimulus interface to instruct or induce the users to perform mental tasks. *Brainstim* provides a convenient method to present stimuli of these paradigms, which is presented in Fig. 3. Firstly, *Brainstim* has encompassed 4 widely-used encoding methods, i.e. oddball encoding for P300 [35], joint frequency-phase modulation for SSVEP [14], upper limb motor imagery for MI [36], and miniature space coding for aVEP [10]. It also allows users to customize settings for paradigm presentation. The class *Experiment()* initializes the main screen, allowing the users to choose a paradigm by the keyboard. And the paradigm classes *P300()*, *SSVEP()*, *MI()*, and *aVEP()* are used to configure the attributes of stimuli, such as the number, size, shape, color, and form. Besides, *Brainstim* provides both parallel and serial ports to communicate with EEG devices for sending event triggers. Fig. 2(d) shows an example of MI paradigms using *Brainstim*.

3.2.3. Brainflow : online real-time framework

Real-time efficiency is vital for online BCI systems. *Brainflow* uses multi-processing and multi-thread techniques to realize a high-speed online framework for BCIs. *Brainflow* has three modules: The online control module *Brainflow.amplifier* is the main process. It contains two threads, one for controlling the EEG device and the other for retrieving online data. And the data processing module *Brainflow.worker* is a subprocess for offline model training and online real-time decoding. The logging module, *Brainflow.logger*, is used to record information about the working process. A usage example of *Brainflow* is shown in Fig. 2(e).

3.3. Basic functions

MetaBCI is a BCI software platform with wide compatibility. It offers flexible and unified Base classes for different datasets, decoding algorithms, encoding paradigms, and EEG recording devices. The functions of MetaBCI can be extended by inheriting these Base classes and overriding certain methods. The user can implement new items as needed, e.g., private datasets, self-designed paradigms, and new algorithms.

3.3.1. Supported public datasets

MetaBCI provides the Base class *BaseDataset* to uniformly reformat the BCI datasets [32]. Creating a dataset in MetaBCI requires its key parameters, i.e., information on subjects, channels, events, sampling rate, and original structure. Based on the above information, the abstract methods in *BaseDataset*, i.e., *_get_single_subject_data()* and *data_path()*, need to be overridden.

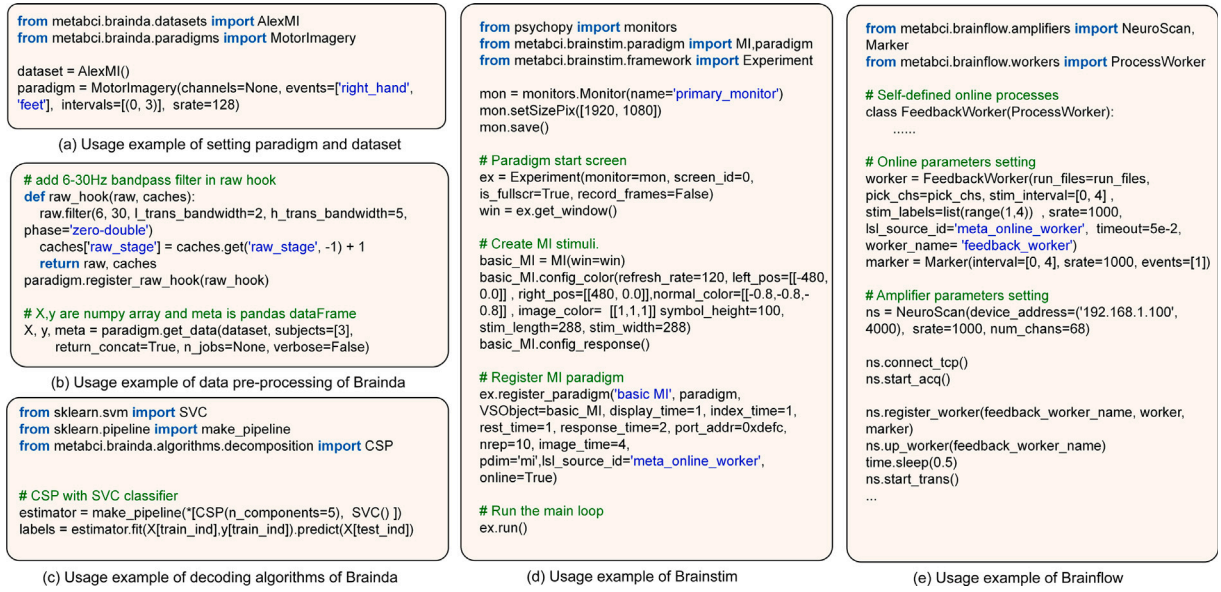


Fig. 2. Example scripts of MetaBCI.

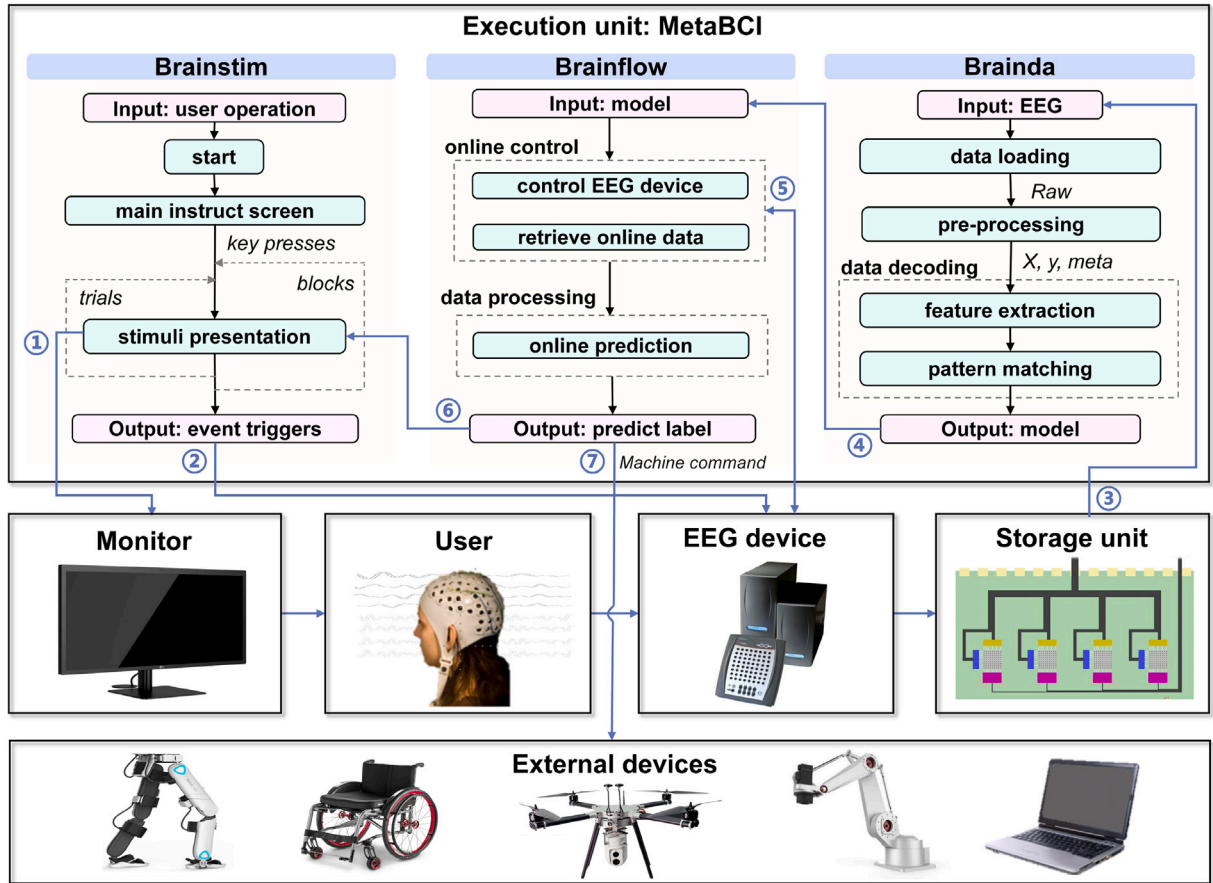


Fig. 3. The main design of MetaBCI. The implications of some arrows are: (1) *Brainstim* presents BCI paradigm stimuli to users; (2) *Brainstim* sends event triggers to EEG devices; (3) *Brainda* trains models using offline data; (4) *Brainflow* will use the model obtained from *Brainda* to perform online prediction; (5) *Brainflow* sends operation commands to the EEG device, and retrieves online data from the EEG device in real time; (6) *Brainflow*'s online prediction results can be fed back into *Brainstim*'s stimulation interface; (7) *Brainflow*'s online prediction results can control external devices.

3.3.2. Supported public datasets

So far, 17 datasets can be accessed in MetaBCI, including AlexMI [37], BNCI2014001 [38], BNCI2014004 [39], Benchmark [40], BETA

[41], Nakanishi2015 [42], Zhou2016 [43], Physionet [44], Cho2017 [45], MunichMI [46], Schirrmeister2017 [47], Weibo2014 [48], CBCIC2019001, CBCIC2019004, CBCIC2020aVep, and Xu2018aVep

[10]. It is worth mentioning that Metabci supports the EEG-BIDS [49] format and provides an example of reading the Matching Pennies dataset [50]. All of them can work with the *Brainda* workflow.

3.3.3. Supported paradigms

MetaBCI provides the Base class *BaseParadigm* in *Brainda* for the needs of realizing different paradigms. It performs a unified pre-processing approach according to user-specified parameters such as channels, intervals, resampling rate, and pre-processing sequences. If these parameters are not provided, *BaseParadigm* will use the default settings. A new paradigm only requires overriding the abstract methods *is_valid()* to validate the match between the dataset and paradigm. Currently, MetaBCI supports 4 commonly used BCI paradigms as examples, i.e., MI [36,51], SSVEP [52,53], P300 [35] and aVEP [10].

3.3.4. Supported decoding algorithms

MetaBCI uses the Base class *BaseEstimator* of Scikit-Learn to standardize BCI decoding algorithms [34]. A new algorithm requires overriding methods *fit()*, *transform()*, and *predict()*. MetaBCI classifies BCI decoding algorithms into four classes according to their mathematical properties [6], including 28 decomposition algorithms, 5 deep learning models, 5 transfer learning models, and 6 Riemannian geometry algorithms. It also contains 6 general processing methods, e.g. filter bank analysis [54], and 3 cross-validation methods.

The traditional decomposition algorithms in BCIs compute spatial filters for feature extraction [6]. The name comes from the fact that most of them can be solved using generalized eigenvalue decomposition (GED). Currently, the performance of EEG-based BCIs relies on the recognition of different EEG features. One of EEG features is the changes in EEG energy generated by evoked brain rhythms, e.g. ERD and ERS for MI-based BCIs [36]. The well-known decomposition methods for energy differences are common spatial pattern (CSP) [55], and its extended versions, i.e., multiclass CSP (MultiCSP) [56], filter bank CSP (FBCSP) [57], and FBMultiCSP. The other one is the time- and phase-locked event-related potentials (ERPs), which have distinct shapes in the time domain, e.g., P300 and SSVEP. For SSVEP-based BCIs, MetaBCI contains 22 decomposition algorithms, including canonical correlation analysis (CCA) [58], task-related component analysis (TRCA) [13], sum of squared correlations (SSCOR) [59], task discriminant component analysis (TDCA) [60], and their extended versions [15,54,61–64]. For P300-based BCIs, MetaBCI provides widely-used shrinkage linear discriminant analysis (SKLDA) [65], spatial-temporal discriminant analysis (STDA) [65], discriminative spatial patterns (DSP) [66], and discriminative canonical pattern matching (DCPM) [10].

Recently, deep learning models have also shown a potential to extract features efficiently in EEG [67]. Therefore, MetaBCI implements 5 deep learning models for BCIs (ShallowNet [68], Deep4Net [68], EEGNet [69], GuueyNet [70], and ConvCA [71]). In addition, Riemannian geometry applies operations on the space of symmetric positive-definite (SPD) matrices. MetaBCI implements 6 Riemannian geometry algorithms, including minimum distance to Riemannian mean (MDRM) [72], fisher geodesic discriminant analysis (FGDA) [72], fisher geodesic MDRM (FgMDRM) [72], tangent space classifier (TSClassifier) [73], Riemannian alignment (RA) [74], and recursive RA (rRA) [75]. Riemannian geometry is usually efficient in MI tasks and can also be applied to extract time-domain features. Another type is transfer learning which learns information across different domains. MetaBCI also reproduces effective cross-domain models, including transfer template-based CCA (ttCCA) [76] and its filter bank version, Riemannian procrustes analysis (RPA) [77], least-squares transformation (LST) [78,79], and manifold embedded knowledge transfer (MEKT) [80]. All decoding methods are implemented uniformly in MetaBCI.

3.3.5. Supported EEG devices

MetaBCI implements the Base class *BaseAmplifier* to support various EEG recording devices. It needs to override the abstract method *recv()* in *BaseAmplifier* based on the communication protocol of the EEG amplifiers. MetaBCI currently supports the online operation of Neuroscan Synamp2 and Neuracle, by providing subclasses *NeuroScan()* and *Neuracle()* respectively. By implementing a communication protocol between MetaBCI and Lab streaming layer (LSL), MetaBCI also indirectly supports devices such as EGI, g.tec, Biosemi, etc.

3.3.6. Supported BCI performance metrics

MetaBCI provides the class *Performance* to evaluate BCI performance. It currently supports 10 evaluation metrics, including 2 BCI-specific metrics, i.e., theoretical information transfer rate (theoretical ITR) and practical information transfer rate (practical ITR), and 8 general machine learning metrics, i.e., accuracy, balanced accuracy, true positive rate (TPR), false negative rate (FNR), false positive rate (FPR), true negative rate (TNR), receiver operating characteristic curve (ROC) and area under curve (AUC). The ITR can be computed as:

$$ITR = (\log_2 M + P \log_2 P + (1 - P) \log_2 \frac{1 - P}{M - 1}) \times \frac{60}{T} \quad (1)$$

where M is the number of commands, P refers to the average accuracy of target classification, and T is the time for target selection in theoretical ITR. For visual BCIs, T is the overall time for both target selection and gaze shift in practical ITR.

4. Result

This section shows four case studies using MetaBCI to conduct offline analysis and online experiments. All the source codes and detailed parameter settings can be found in the ‘demos’ folder of the MetaBCI repository. The accuracy and practical ITR were calculated for evaluating the MetaBCI performance.

4.1. SSVEP data decoding

In this case, we took the public dataset Benchmark [40] as an example. This dataset contained 35 subjects’ data collected from a cue-guided experiment. For each subject, the experiment included 6 blocks and each block contained 40 trials corresponding to 40 targets. The data length of each trial was 6 s constituted by 0.5 s for the target cue, 5 s for the visual stimulation, and 0.5 s for the rest blank. The raw data of each subject was a 4-D matrix and was saved in MAT format. Here, filter bank TRCA (FBTRCA), filter bank TDCA (FBTDCA), and filter bank extended CCA (FBeCCA) were tested on the dataset against different data lengths ranging from 0.5 s to 1.5 s. The accuracy and ITR were computed under the leave-one-out cross-validation. One block was used as test data, and the remaining blocks were used as training data. We selected the test data in turn, and repeated this 6 times.

The decoding accuracy and ITR are shown in Fig. 4. For both accuracy and ITR, FBTDCA performed the highest among the three methods. For all methods, the classification accuracy increased with the data length increased. The highest accuracies of FBTDCA, FBTRCA, and FBeCCA were $98.40 \pm 0.45\%$, $96.90 \pm 1.03\%$, and $96.62 \pm 1.07\%$, respectively. The ITR showed a trend of rising first and then falling. FBTDCA, FBTRCA, and FBeCCA reached the highest ITR when the data lengths were 0.7 s, 0.8 s, and 0.9 s, corresponding to 101.28 ± 3.64 bits/min, 97.13 ± 3.76 bits/min, and 94.81 ± 3.33 bits/min, respectively. The results suggested that the SSVEP decoding methods provided by MetaBCI were able to detect SSVEP components efficiently.

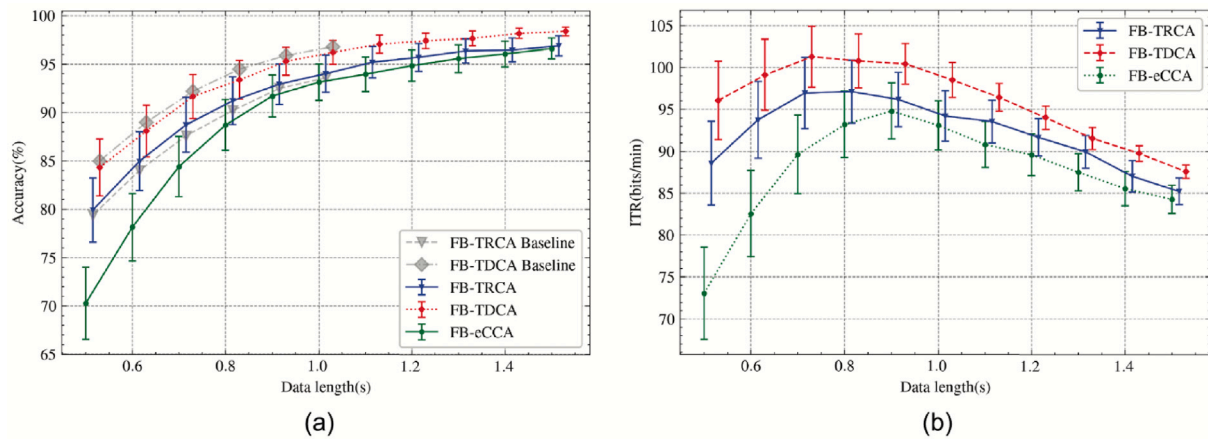


Fig. 4. (a) Classification accuracy for FBTDCA, FBTRCA, and FBeCCA methods on Benchmark dataset against different data lengths range from 0.5 s to 1.5 s with a step of 0.1 s. Gray lines are the baseline from [60]. (b) ITRs of these methods. The gaze switching duration for computing the ITR was 2 s. The error bars indicate the standard errors.

Table 1
Classification accuracy of different MI decoding methods on AlexMI dataset.

	S1	S2	S3	S4	S5	S6	S7	S8	Mean.
FB-CSP	62.50%	67.50%	87.50%	57.50%	60.00%	47.50%	95.00%	70.00%	68.44%
FB-multiCSP	51.67%	58.33%	65.00%	63.33%	38.33%	35.00%	65.00%	66.67%	55.42%
Deep4Net	75.00%	80.00%	82.50%	70.00%	67.50%	60.00%	87.50%	82.50%	75.63%
EEGNet	92.50%	77.50%	87.50%	85.00%	85.00%	57.50%	90.00%	77.50%	81.56%
ShallowNet	72.50%	60.00%	75.00%	67.50%	80.00%	65.00%	82.50%	87.50%	73.75%

4.2. MI data decoding

In this case, the dataset AlexMI [37] was used. This dataset contained EEG recordings from 8 subjects, performing 3 tasks including right-hand motor imagination, feet motor imagination, and rest. The duration of a single trial was 3 s, and there were 20 trials for each class. Data were collected at 512 Hz with 16 wet electrodes and saved in MNE raw file format. FBCSP, FBmultiCSP, Deep4Net, EEGNet, and ShallowNet were tested on this dataset. The filter bank parameters applied in CSP and multiCSP were the same. The accuracy corresponding to each method was computed under 5-fold cross-validation and the random seed for splitting the train and test set was kept the same for all methods. The results of classification accuracy are listed in Table 1. The deep learning methods (i.e., Deep4Net, EEGNet, and ShallowNet) showed better accuracy than CSP-based methods. For the deep learning methods, the EEGNet reached the highest average accuracy which was 81.56%.

4.3. aVEP data decoding

This study used a miniature aVEP dataset collected in the previous study [10]. Twelve subjects participated in the offline aVEP speller experiment. The lateral visual stimuli consisted of six tiny dots gathered within 0.5° of the visual angle and located at an eccentricity of 2.1° . The left-right stimulus sequence was coded '0', while the right-left stimulus was coded '1'. Thirty-two different code sequences were created using 5 bits in this study, which were arbitrarily allocated to different characters. Six repetitions of the stimulus sequences were used in the offline experiment. Each subject completed 3 rounds of character spelling tasks and each round iterated through all 32 characters in random order. The algorithm DCPM [10] was used to recognize different patterns of aVEPs. The decoding accuracy was estimated by leave-one-out cross-validation in which 2 rounds of data were used for training and 1 round was used for testing. The basic encoding unit ('0' and '1') classification accuracy ranged from 67.91% to 88.61% with an average of 76.99%. The character classification accuracy of the miniature aVEP speller ranged from 47.91% to 90.63% with an average of 72.65% when averaging decision values over 6 repetitions.

4.4. Online experiments

Online cue-guided experiments of SSVEP, P300, and MI were conducted to validate the feasibility of *Brainstim* and *Brainflow*. All of them included a training phase and a testing phase. The feedback results would be presented on the screen in the testing phase. The paradigms were presented on a 27-inch LCD monitor. The refresh rate and resolution of the monitor were 240 Hz and $2,160 \times 1,440$ pixels, respectively. EEG signals were recorded using Neuroscan Synamp2 at a sampling rate of 1000 Hz, and then down-sampled to 256 Hz. The electrode location followed the international 10/20 system. Three subjects with corrected-to-normal vision participated in the experiments respectively (2 females, mean age: 23 years). The Institutional Review Board at Tianjin University approved the experimental protocol. According to the Helsinki Declaration, all subjects were fully informed of all procedures, understood all possible consequences of the study, and signed an informed consent form.

In the SSVEP experiment, a 4×5 character matrix was presented on the screen, as shown in Fig. 5(a). The frequency range was 8 to 15.8 Hz with a 0.4 Hz interval, and the phase changed with a 0.35π interval. In each block, all targets would be selected once at random. There were 0.5 s cue time and 0.5 s flicker time for a trial. The subject performed 6 blocks for training and 2 blocks for testing. Nine occipital channels (PO3/4, PO5/6, POz, Pz, O1/2, and Oz) were chosen for online experiments. The algorithm FBTDCA was used for data decoding [60].

In the P300 experiment, a row/column speller featuring a 6×6 character matrix was presented on the screen, as shown in Fig. 5(b). For each round, all rows and columns were randomly lightened for 100 ms and extinguished for 75 ms. Six rounds were conducted for a target. All targets would be selected in a block. The subject performed 3 blocks for the training and 2 blocks for the testing. For the online experiments, six channels (FCz, Cz, Pz, PO7/8, and Oz) were chosen. The algorithm LDA was used for data decoding [34].

In the MI experiment, the subject performed the task of upper limb motor imagery, as shown in Fig. 5(c). After 1 s cue time, the image of the left or right hand appeared on the screen and lasted 4 s. It was

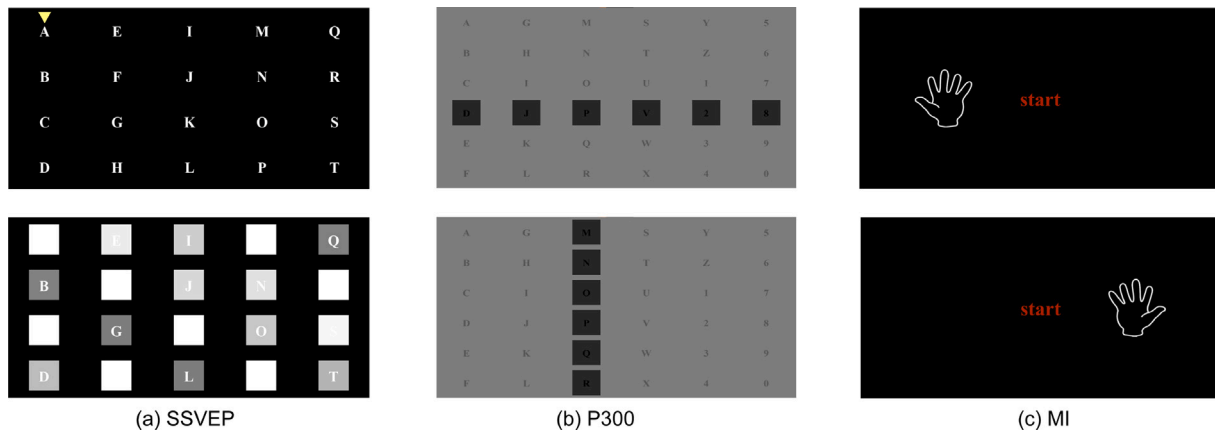
Fig. 5. Paradigm presentation for cue-guided experiments using *Brainstim*.

Table 2

Performance of online experiments using MetaBCI.

Paradigm	Offline accuracy	Online accuracy
SSVEP	99.0%	100.0%
P300	88.9%	91.7%
MI	98.3%	94.4%

Table 3

Time delay estimation of Brainflow.

Time delay	Data communication	Results feedback
Core i7-12700KF	86.95 ms	0.39 ms
Xeon 6226R	78.19 ms	0.60 ms
Core i9-12900K	67.61 ms	0.40 ms

repeated 15 times in a block at random. The subject performed 4 blocks for training and 3 blocks for testing. Twenty-eight channels (FC5/6, FC3/4, FC1/2, FCZ, C5/6, C3/4, C1/2, Cz, CP5/6, CP3/4, CP1/2, CPz, P5/6, P3/4, P1/2, and Pz) were chosen for online experiments. Data decoding was performed using FBCSP for feature extraction and the support vector machine (SVM) for pattern matching [57].

The photo-transistor was used to test the validity of stimuli generated by *Brainstim*. The photo-transistor validation was conducted using an ambient light sensor TEMENT6000. The stimuli in the BCI paradigm were presented on the same 27-inch LCD monitor as in the online experiment. The sensor was positioned approximately 1 cm in front of one of the stimuli. Then, the sensor collected light signals and converted them into electrical signals, which were recorded offline through the signal acquisition device. Fig. 6(a) shows the photo-transistor signals of 12 Hz/1.5 π SSVEP stimuli. The signals of different trials in both time and frequency domains had similar shapes, and the peak of the spectrum appeared at 12 Hz. Fig. 6(b) illustrates the photo-transistor signals of P300 stimuli (row 2, column 6). In each round, the stimuli generated a change in brightness at the target labels. The average time delays were 13.73 ± 0.73 ms and 4.58 ± 0.60 ms, for SSVEP and P300 stimuli, respectively. The results indicate that stimuli generated by *Brainstim* are stable and effective.

This study analyzed the characteristics of SSVEP in the frequency domain, P300 in the time domain, and MI in the spatial domain, respectively. As seen in Fig. 7(a), amplitude peaks of average SSVEP could be observed at 12 Hz and its harmonic frequencies (12 Hz: 1.86 μ V; 24 Hz: 3.31 μ V; 36 Hz: 0.84 μ V). Fig. 7(b) shows the target, non-target, and their differential ERPs induced by P300 speller. The target stimuli induced a larger P300 potential than the non-target stimuli at about 300 ms after stimulus onset. The ERD of MI could be clearly observed in Fig. 7(c). It had the phenomenon of contralateral dominance.

Offline accuracy in the training phase and online accuracy in the testing phase were calculated, as listed in Table 2. The offline accuracies for SSVEP, P300, and MI were 99.0%, 88.9%, and 98.3%, respectively, and online accuracies were 100.0%, 91.7%, and 94.4%, respectively.

We also tested the time delay of *Brainflow* on computers with different hardware configurations, as shown in Table 3. The time delay

for data communication is the time difference between sending the stimulus label and acquiring the single-trial EEG online, and then excluding the signal duration. The time delay for result feedback is the time required from online recognition to feedback results. The results showed similar time delays using different devices. It proved that the online real-time framework *Brainflow* was insensitive to computer configuration and had wide applicability.

5. Discussion

5.1. Significance of MetaBCI

MetaBCI is a feature-rich software platform that covers all links of the BCI chain. It helps users to focus on a BCI link of interest without knowing the technical details of the other links. *Brainda* is designed to lower the technical threshold of offline analysis in BCI studies. It supports the auto-download of multiple public datasets, which saves time in searching and downloading them. Also, *Brainda* provides externally configured *hook()* to make data preprocessing more flexible, and raw data can be processed into a unified format for offline analysis. The decoding pipeline in *Brainda* follows the widely-used approaches of machine learning, i.e., *fit()*, *transform()*, and *predict()*. Users can easily access different decoding algorithms in the same way without changing the source codes. *Brainstim* is a module specifically designed to present stimuli for BCI paradigms. It is easy and user-friendly since users only need to set the initial parameters of paradigms. Meanwhile, the stimuli generated by *Brainstim* are stable and precise, as seen in Fig. 6. *Brainflow* is a framework for building up online BCI systems. It uses multi-processing and multi-thread techniques to work on real-time data streams, boosting the speed and efficiency of the online system. Additionally, the offline model trained in *Brainda* can be used directly in *Brainflow* for online decoding. In general, by combining *Brainda*, *Brainstim*, and *Brainflow*, users can easily establish a BCI system with all links using MetaBCI.

From the perspective of code execution, MetaBCI is programmed with the internationally popular open-source language Python. Many toolkits have been developed based on it in various fields, which facilitates cross-field research between BCI and other advanced technologies. It makes MetaBCI easy to implement and provides rich technical support to extend its functions. Furthermore, Python is an easy-to-use

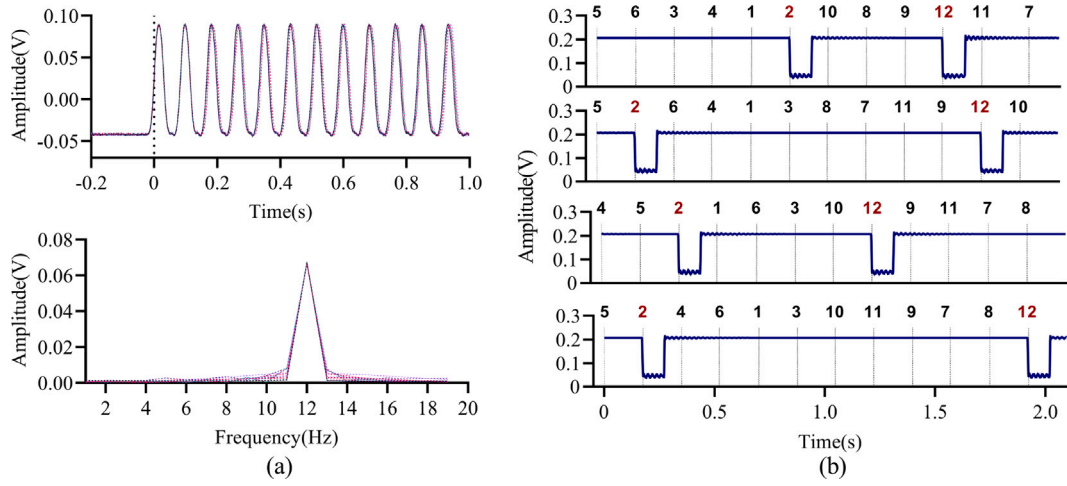


Fig. 6. Results of photo-transistor testing. (a) The photo-transistor signals in the time (top) and frequency (bottom) domains of 12 Hz/1.5 π SSVEP stimuli. The lines of different colors represent different trials. (b) The photo-transistor signals of P300 stimuli (row 2, column 6). From top to bottom are different rounds. The numbers above the gray dashed lines represent the corresponding labels. The target labels are 2 and 12, marked in red.

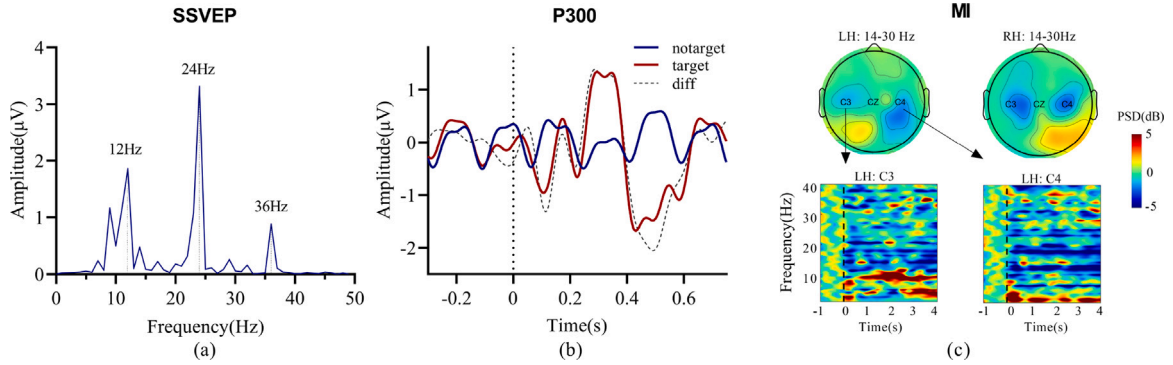


Fig. 7. Data analysis of online experiments. (a) The average 12 Hz SSVEP from OZ electrode in the frequency domain. The dotted lines represent its fundamental and harmonic frequencies. (b) The target, non-target, and their differential ERPs from CZ electrode, induced by P300 speller. (c) The spatial distribution of average MI signals corresponding to left ('LH') and right hand ('RH') movements, and the averaged time-frequency maps on C3 and C4 for left-hand movements.

programming language that allows the user to review and edit the source scripts of MetaBCI according to their research and application requirements. It can significantly reduce the repeated work in BCIs, thus saving the time for developers to perform data encoding, loading, processing, and online feedback.

However, for BCI professionals, repeating existing work is not sufficient to develop new technologies. To this end, MetaBCI also provides extensible Base classes to satisfy high-level secondary development requirements (See Section 3.3). It is a code framework that covers all links of BCIs, and is easily compatible with new datasets, paradigms, algorithms, and devices. Meanwhile, MetaBCI repository is hosted on Github, where all developers can also submit issues and start pull requests for adding new features. MetaBCI expects to receive new contributions from the BCI community. It will promote the iteration and dissemination of advanced BCI technologies.

MetaBCI also has the potential to become a benchmark for BCI research. In many fields such as computer vision, reinforcement learning, and natural language processing, there are many benchmarks provided by open-source organizations for algorithm performance comparison, which strongly boosted the development of relevant fields. The BCI community also expects a benchmark for applying and comparing state-of-the-art algorithms, which is a basic step for making progress. MOABB [32] is a commonly-used benchmark in the BCI field and provides an efficient approach to work with datasets in different formats. Compared to MOABB [32], MetaBCI offers hook functions to make the preprocessing flow easier, the *Meta* variable to contain necessary

information about EEG data and the latest EEG decoding algorithms. More notably, because all BCI chains are covered, MetaBCI is able to be used as a benchmark to assess the performance of the entire BCI system, not only the EEG decoding link from, a more practical view, e.g., online performance. In the future, more efforts will be made to improve the functions of MetaBCI and thus make it a general benchmark.

5.2. Relationship and comparison with other toolkits

Currently, several toolkits for the development of BCIs are available for researchers, as listed in Table 4. MOABB [32] is a trustworthy algorithm benchmark for BCIs. It provides a platform for offline comparison of decoding algorithms. LSL [26] offers many apps for reading, recording, and synchronizing data streams from multiple devices, which makes it easy to start experiments using new devices without knowing protocol details. However, none of these toolkits can cover all the links of BCI. The toolkit BCI2000 [27] contains the whole workflow of paradigm design, data collection, and data analysis, but it only provides developing interfaces to these links without implementing specific datasets, algorithms, paradigms, etc. Moreover, it is developed based on C++, a hard-to-learn programming language, which makes it difficult for researchers to conduct secondary development. As far as our knowledge extends, there is currently no Python-based open-source software platforms that comprehensively encompasses all the links within a BCI system.

Compared with the existing toolkits, MetaBCI focuses on realizing higher-level applications (i.e., stimuli presentation, offline pipelines,

Table 4
Reviews on existing BCI toolkits.

	Toolkit	Open sourced	Programming language	Application context				Features
				Paradigm design	Datasets benchmark	Data analysis	BCI algorithms	Data collection
1	Psychtoolbox	Yes	Matlab	✓				Support sub-millisecond timing; Support low-latency visual and audio stimuli.
2	Psychopy	Yes	Python	✓				Provide a drag and drop interface; Based on Python.
3	E-prime	No	E-basic	✓				✓ Provide interface for drag and drop design; Millisecond-accurate visual and audio stimuli.
4	MNE-Python	Yes	Python			✓		Support multi neurophysiology data analysis; Multi types of analysis methods supported.
5	EEGLAB	Yes	Matlab			✓		Multiple EEG special data analysis functions; Support to import multiple formats EEG data.
6	Fieldtrip	Yes	Matlab			✓		Support to analyze MEG, EEG and iEEG; Both support animal and human data.
7	Brainstorm	Yes	Matlab			✓		Support both invasive and noninvasive BCI data; Graphical interface provided.
8	MOABB	Yes	Python		✓	✓	✓	Provide unified way to format dataset; Provide algorithm comparison metrics.
9	Lab streaming layer	Yes	C++					✓ Provide apps for support multiple devices; Provide interface for secondary development.
10	BCI2000	Yes	C++	✓		✓		✓ Manage data I/O in common file formats; Support whole workflow for BCI experiment.
11	OpenVibe	Yes	Matlab Python	✓		✓		✓ Support for various acquisition devices; Provide basic classification method.
12	MetaBCI	Yes	Python	✓	✓	✓	✓	✓ Covers all links of the BCI chain.

and online workflows). Thus, MetaBCI grows from the existing toolkits, especially the toolkits that were programmed in Python. For example, the front end of *Brainstim* was developed based on Psychopy-coder [25], which provides various functions for drawing graphics and controlling the experimental tasks. The design of *Brainda* is inspired by MOABB [32] and follows its basic strategy of dataset formatting. We utilize several functions of MNE-python [29] for data loading, formatting, and pre-processing, and the algorithm interface of Scikit-learn [34] for data decoding. Also, *Brainflow* indirectly supports some EEG acquisition devices using a communication protocol between MetaBCI and LSL [26]. In summary, MetaBCI benefits from the open-source community and will adhere to the idea of open-source.

5.3. Limitations and future development plans

MetaBCI has facilitated advancements in the field of BCI, yet opportunities for improvement remain. One key limitation pertains to the insufficiency of data analysis and decoding methods. The intricacies of EEG signals, characterized by non-linearity, non-stationarity, and substantial variability, demand an expansion of advanced EEG analysis and decoding methods. To address this, we intend to include more advanced EEG analysis and decoding methods, e.g. explainable artificial intelligence (XAI) models [81], enabling researchers to gain a deeper understanding of EEG features and enhance BCI research. Additionally, the current need for external tools to support various EEG devices poses inconveniences. In the future, MetaBCI will offer direct support for a broader range of EEG devices and compatibility with other BCI-related equipment. Furthermore, to diversify its applications, MetaBCI will implement examples of BCI driving external devices, fostering practical usage in scenarios like real-time brain-controlled drone systems. These measures constitute our commitment to comprehensively advance the MetaBCI platform, embracing contributions from the open-source community for the collective benefit of BCI research.

6. Conclusion

This study proposes a novel open-source software platform in Python, i.e., MetaBCI, to facilitate the implementation of all BCI links including stimulus presentation, data collection and processing, and command feedback. The functionality of MetaBCI is validated by 3

offline cases and 3 online cases. The results show that MetaBCI is an effective and simple-to-use software platform for BCI research and application. MetaBCI is feature-rich and highly extensible, with the potential to become a BCI benchmark platform. It has both methodological and practical implications and is significant in pushing the boundary of BCI towards real-world applications.

CRedit authorship contribution statement

Jie Mei: Data curation, Investigation, Methodology, Software, Visualization, Writing – original draft, Writing – review & editing. **Ruixin Luo:** Investigation, Methodology, Resources, Software, Validation, Visualization, Writing – original draft, Writing – review & editing. **Lichao Xu:** Investigation, Software. **Wei Zhao:** Methodology. **Shengfu Wen:** Formal analysis. **Kun Wang:** Project administration, Validation. **Xi-aolin Xiao:** Validation. **Jiayuan Meng:** Validation. **Yongzhi Huang:** Validation. **Jiabei Tang:** Resources. **Longlong Cheng:** Resources. **Min-peng Xu:** Conceptualization, Funding acquisition, Supervision, Writing – review & editing. **Dong Ming:** Conceptualization, Funding acquisition, Project administration, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by the National Key Research and Development Program of China (Grant No. 2022YFF1202300), National Natural Science Foundation of China (No. 62122059, 61976152, 81925020, 62206198, 62106170, 62106173, 82101274), Introduce Innovative Teams of 2021 “New High School 20 Items” Project (2021GXRC071). The authors sincerely thank all contributors of MetaBCI.

References

- [1] J.J. Vidal, Toward direct brain-computer communication, *Ann. Rev. Biophys. Bioeng.* 2 (1) (1973) 157–180.
- [2] J. Wolpaw, N. Birbaumer, W. Heetderks, D. McFarland, P. Peckham, G. Schalk, E. Donchin, L. Quatrano, C. Robinson, T. Vaughan, Brain-computer interface technology: a review of the first international meeting, *IEEE Trans. Rehabil. Eng.* 8 (2) (2000) 164–173.
- [3] J.R. Wolpaw, N. Birbaumer, D.J. McFarland, G. Pfurtscheller, T.M. Vaughan, Brain-computer interfaces for communication and control, *Clin. Neurophysiol.* (2002) 25.
- [4] J. Ju, A.G. Feleke, L. Luo, X. Fan, Recognition of drivers' hard and soft braking intentions based on hybrid brain-computer interfaces, *Cyborg Bionic Syst.* 2022 (2022).
- [5] M. Rashid, N. Sulaiman, A. P. P. Abdol Majeed, R.M. Musa, A.F. Ab. Nasir, B.S. Bari, S. Khatun, Current status, challenges, and possible solutions of EEG-based brain-computer interface: A comprehensive review, *Front. Neurobot.* 14 (2020) 25.
- [6] L. Xu, M. Xu, T.P. Jung, D. Ming, Review of brain encoding and decoding mechanisms for EEG-based brain-computer interface, *Cogn. Neurodyn.* 15 (5) (2021) 921.
- [7] L. Xu, M. Xu, Z. Ma, K. Wang, T.-P. Jung, D. Ming, Enhancing transfer performance across datasets for brain-computer interfaces using a combination of alignment strategies and adaptive batch normalization, *J. Neural Eng.* 18 (4) (2021) 0460e5.
- [8] J. Meng, T. Streitz, N. Gulachek, D. Suma, B. He, Three-dimensional brain-computer interface control through simultaneous overt spatial attentional and motor imagery tasks, *IEEE Trans. Biomed. Eng.* 65 (11) (2018) 2417–2427.
- [9] K. Wang, M. Xu, Y. Wang, S. Zhang, L. Chen, D. Ming, Enhance decoding of pre-movement EEG patterns for brain-computer interfaces, *J. Neural Eng.* 17 (1) (2020) 016033.
- [10] M. Xu, X. Xiao, Y. Wang, H. Qi, T.P. Jung, D. Ming, A brain-computer interface based on miniature-event-related potentials induced by very small lateral visual stimuli, *IEEE Trans. Biomed. Eng.* 65 (5) (2018) 1166–1175.
- [11] M. Xu, J. Han, Y. Wang, T.-P. Jung, D. Ming, Implementing over 100 command codes for a high-speed hybrid brain-computer interface using concurrent P300 and SSVEP features, *IEEE Trans. Biomed. Eng.* (2020) 1.
- [12] Z. Zhao, Y. Lin, Y. Wang, X. Gao, Single-trial EEG classification using spatio-temporal weighting and correlation analysis for RSVP-based collaborative brain computer interface, *IEEE Trans. Biomed. Eng.* (2023) 1–9.
- [13] M. Nakanishi, Y. Wang, X. Chen, Y.T. Wang, X. Gao, T.P. Jung, Enhancing detection of SSVEPs for a high-speed brain speller using task-related component analysis, *IEEE Trans. Biomed. Eng.* 65 (1) (2018) 104–112.
- [14] X. Chen, Y. Wang, M. Nakanishi, X. Gao, T.-P. Jung, S. Gao, High-speed spelling with a noninvasive brain-computer interface, *Proc. Natl. Acad. Sci.* 112 (44) (2015) E6058–E6067.
- [15] R. Luo, M. Xu, X. Zhou, X. Xiao, T.-P. Jung, D. Ming, Data augmentation of SSVEPs using source aliasing matrix estimation for brain-computer interfaces, *IEEE Trans. Biomed. Eng.* (2022) 1–10.
- [16] X. Xiao, L. Xu, J. Yue, B. Pan, M. Xu, D. Ming, Fixed template network and dynamic template network: novel network designs for decoding steady-state visual evoked potentials, *J. Neural Eng.* 19 (5) (2022) 056049.
- [17] J. Han, M. Xu, X. Xiao, W. Yi, T.-P. Jung, D. Ming, A high-speed hybrid brain-computer interface with more than 200 targets, *J. Neural Eng.* 20 (1) (2023) 016025.
- [18] K. LaFleur, K. Cassady, A. Doud, K. Shades, E. Rogin, B. He, Quadcopter control in three-dimensional space using a noninvasive motor imagery-based brain-computer interface, *J. Neural Eng.* 10 (4) (2013) 046003.
- [19] J. Mei, M. Xu, L. Wang, Y. Ke, Y. Wang, T.-P. Jung, D. Ming, Using SSVEP-BCI to continuous control a quadcopter with 4-DOF motions, in: 2020 42nd Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC), 2020, pp. 4745–4748, <http://dx.doi.org/10.1109/EMBC44109.2020.9176131>.
- [20] B.J. Edelman, J. Meng, D. Suma, C. Zurn, E. Nagarajan, B.S. Baxter, C.C. Cline, B. He, Noninvasive neuroimaging enhances continuous neural tracking for robotic device control, *Science Robotics* 4 (31) (2019) eaaw6844.
- [21] J. Meng, S. Zhang, A. Bekyo, J. Olsoe, B. Baxter, B. He, Noninvasive electroencephalogram based control of a robotic arm for reach and grasp tasks, *Sci. Rep.* 6 (1) (2016) 38565.
- [22] Y. Li, J. Pan, F. Wang, Z. Yu, A hybrid BCI system combining P300 and SSVEP and its application to wheelchair control, *IEEE Trans. Biomed. Eng.* 60 (11) (2013) 3156–3166.
- [23] M. Xu, F. He, T.-P. Jung, X. Gu, D. Ming, Current challenges for the practical application of electroencephalography-based brain-computer interfaces, *Engineering* 7 (12) (2021) 1710–1712.
- [24] D.H. Brainard, The psychophysics toolbox, *Spatial Vis.* 10 (4) (1997) 433–436.
- [25] J. Pearce, J.R. Gray, S. Simpson, M. MacAskill, R. Höchenberger, H. Sogo, E. Kastman, J.K. Lindelöv, PsychoPy2: Experiments in behavior made easy, *Behav. Res. Methods* 51 (1) (2019) 195–203.
- [26] T. Stenner, C. Boulay, M. Grivich, D. Medine, C. Kothe, tobiascherzke, G. Grimm, xloem, A. Biancarelli, B. Mansencal, chausner, J. Frey, kyucrane, S. Powell, P. Clisson, phfix, Scvn/liblsl: v1.16.0, 2022, <http://dx.doi.org/10.5281/zenodo.6387090>, URL <https://zenodo.org/record/6387090>.
- [27] G. Schalk, D. McFarland, T. Hinterberger, N. Birbaumer, J. Wolpaw, BCI2000: a general-purpose brain-computer interface (BCI) system, *IEEE Trans. Biomed. Eng.* 51 (6) (2004) 1034–1043.
- [28] A. Delorme, S. Makeig, EEGLAB: an open source toolbox for analysis of single-trial EEG dynamics including independent component analysis, *J. Neurosci. Methods* 134 (1) (2004) 9–21.
- [29] A. Gramfort, M. Luessi, E. Larson, D.A. Engemann, D. Strohmeier, C. Brodbeck, L. Parkkonen, M.S. Hämäläinen, MNE software for processing MEG and EEG data, *NeuroImage* 86 (2014) 446–460.
- [30] R. Oostenveld, P. Fries, E. Maris, J.-M. Schoffelen, FieldTrip: Open source software for advanced analysis of MEG, EEG, and invasive electrophysiological data, *Comput. Intell. Neurosci.* 2011 (2011) 1–9.
- [31] F. Tadel, S. Baillet, J.C. Mosher, D. Pantazis, R.M. Leahy, Brainstorm: A user-friendly application for MEG/EEG analysis, *Comput. Intell. Neurosci.* 2011 (2011) 1–13.
- [32] V. Jayaram, A. Barachant, MOABB: trustworthy algorithm benchmarking for BCIs, *J. Neural Eng.* 15 (6) (2018) 066011.
- [33] Y. Renard, F. Lotte, G. Gibert, M. Congedo, E. Maby, V. Delannoy, O. Bertrand, A. Lécuyer, OpenViBE: An open-source software platform to design, test, and use brain-computer interfaces in real and virtual environments, *Presence* 19 (1) (2010) 35–53.
- [34] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, E. Duchesnay, Scikit-learn: Machine learning in python, *J. Mach. Learn. Res.* 12 (2011) 2825–2830.
- [35] L. Citi, R. Poli, C. Cinel, F. Sepulveda, P300-based BCI mouse with genetically-optimized analogue control, *IEEE Trans. Neural Syst. Rehabil. Eng.* 16 (1) (2008) 51–61.
- [36] P. Rithwik, V.K. Benzy, A.P. Vinod, High accuracy decoding of motor imagery directions from EEG-based brain computer interface using filter bank spatially regularised common spatial pattern method, *Biomed. Signal Process. Control* 72 (2022).
- [37] A. Barachant, Commande Robuste D'un Effecteur Par Une Interface Cerveau Machine EEG Asynchrone (Theses), (2012)GREN112 Université de Grenoble, 2012, URL <https://tel.archives-ouvertes.fr/tel-01196752>.
- [38] M. Tangermann, K.R. Muller, A. Aertsen, N. Birbaumer, C. Braun, C. Brunner, R. Leeb, C. Mehring, K.J. Müller, G.R. Müller-Putz, G. Nolte, G. Pfurtscheller, H. Preissl, G. Schalk, A. Schlögl, C. Vidaurre, S. Waldert, B. Blankertz, Review of the BCI competition IV, *Front. Neurosci.* 6 (2012) 55.
- [39] R. Leeb, F. Lee, C. Keinrath, R. Scherer, H. Bischof, G. Pfurtscheller, Brain-computer communication: motivation, aim, and impact of exploring a virtual apartment, *IEEE Trans. Neural Syst. Rehabil. Eng.* 15 (4) (2007) 473–482.
- [40] Y. Wang, X. Chen, X. Gao, S. Gao, A benchmark dataset for SSVEP-based brain-computer interfaces, *IEEE Trans. Neural Syst. Rehabil. Eng.* 25 (10) (2017) 1746–1752.
- [41] B. Liu, X. Huang, Y. Wang, X. Chen, X. Gao, BETA: A large benchmark database toward SSVEP-BCI application, *Front. Neurosci.* 14 (2020) 627.
- [42] M. Nakanishi, Y. Wang, Y.T. Wang, T.P. Jung, A comparison study of canonical correlation analysis based methods for detecting steady-state visual evoked potentials, *PLoS One* 10 (10) (2015) e0140703.
- [43] B. Zhou, X. Wu, Z. Lv, L. Zhang, X. Guo, A fully automated trial selection method for optimization of motor imagery based brain-computer interface, *PLoS One* 11 (9) (2016) e0162657.
- [44] A. Goldberger, L. Amaral, L. Glass, J. Hausdorff, P. Ivanov, R. Mark, J. Mietus, G. Moody, C.-K. Peng, H. Stanley, PhysioToolkit, and PhysioNetComponents of a new research resource for complex physiologic signals, *Circulation: J. Am. Heart Assoc.* 101 (23) (2000) 215–220.
- [45] H. Cho, M. Ahn, S. Ahn, M. Kwon, S.C. Jun, EEG datasets for motor imagery brain-computer interface, *Gigascience* 6 (7) (2017) 1–8.
- [46] M. Grosse-Wentrup, C. Liefhold, K. Gramann, M. Buss, Beamforming in non-invasive brain-computer interfaces, *IEEE Trans. Biomed. Eng.* 56 (4) (2009) 1209–1219.
- [47] R.T. Schirrmester, J.T. Springenberg, L.D.J. Fiederer, M. Glasstetter, K. Eggenberger, M. Tangermann, F. Hutter, W. Burgard, T. Ball, Deep learning with convolutional neural networks for EEG decoding and visualization, *Hum. Brain Mapp.* 38 (11) (2017) 5391–5420.
- [48] W. Yi, S. Qiu, K. Wang, H. Qi, L. Zhang, P. Zhou, F. He, D. Ming, Evaluation of EEG oscillatory patterns and cognitive process during simple and compound limb motor imagery, *PLoS One* 9 (12) (2014) e114853.
- [49] C.R. Pernet, S. Appelhoff, K.J. Gorgolewski, G. Flandin, C. Phillips, A. Delorme, R. Oostenveld, EEG-BIDS, an extension to the brain imaging data structure for electroencephalography, *Sci. Data* 6 (1) (2019) 103.
- [50] S. Appelhoff, D. Sauer, S.S. Gill, Matching pennies: A brain computer interface implementation dataset, 2018.
- [51] A. Echtioui, W. Zouch, M. Ghorbel, C. Mhiri, H. Hamam, Multi-class motor imagery EEG classification using convolution neural network, in: Special Session on Super Distributed and Multi-Agent Intelligent Systems, 2021.

- [52] X. Chen, Y. Wang, M. Nakanishi, X. Gao, T.P. Jung, S. Gao, High-speed spelling with a noninvasive brain-computer interface, *Proc. Natl. Acad. Sci. USA* 112 (44) (2015) E6058–67.
- [53] X.T. Wang, A.P. Liu, L. Wu, C. Li, Y. Liu, X. Chen, A generalized zero-shot learning scheme for SSVEP-based BCI system, *Ieee Trans. Neural Syst. Rehabil. Eng.* 31 (2023) 863–874.
- [54] X. Chen, Y. Wang, S. Gao, T.P. Jung, X. Gao, Filter bank canonical correlation analysis for implementing a high-speed SSVEP-based brain-computer interface, *J. Neural Eng.* 12 (4) (2015) 046008.
- [55] H. Ramoser, J. Muller-Gerking, G. Pfurtscheller, Optimal spatial filtering of single trial EEG during imagined hand movement, *IEEE Trans. Rehabil. Eng.* 8 (4) (2000) 441–446.
- [56] M. Grosse-Wentrup, M. Buss, Multiclass common spatial patterns and information theoretic feature extraction, *IEEE Trans. Biomed. Eng.* 55 (8) (2008) 1991–2000.
- [57] K.K. Ang, Z.Y. Chin, H. Zhang, C. Guan, Filter bank common spatial pattern (FBCSP) in brain-computer interface, in: 2008 IEEE International Joint Conference on Neural Networks (IEEE World Congress on Computational Intelligence), 2008, pp. 2390–2397, <http://dx.doi.org/10.1109/IJCNN.2008.4634130>.
- [58] Z. Lin, C. Zhang, W. Wu, X. Gao, Frequency recognition based on canonical correlation analysis for SSVEP-based BCIs, *IEEE Trans. Biomed. Eng.* 53 (12) (2006) 2610–2614.
- [59] G.R. Kiran Kumar, M. Ramasubba Reddy, Designing a sum of squared correlations framework for enhancing SSVEP-based BCIs, *IEEE Trans. Neural Syst. Rehabil. Eng.* 27 (10) (2019) 2044–2050.
- [60] B. Liu, X. Chen, N. Shi, Y. Wang, S. Gao, X. Gao, Improving the performance of individually calibrated SSVEP-BCI by task- discriminant component analysis, *IEEE Trans. Neural Syst. Rehabil. Eng.* 29 (2021) 1998–2007.
- [61] G. Bin, X. Gao, Y. Wang, Y. Li, B. Hong, S. Gao, A high-speed BCI based on code modulation VEP, *J. Neural Eng.* 8 (2) (2011) 025015.
- [62] Y. Zhang, G. Zhou, J. Jin, X. Wang, A. Cichocki, Frequency recognition in SSVEP-based BCI using multiset canonical correlation analysis, *Int. J. Neural Syst.* 24 (4) (2014) 1450013.
- [63] C.M. Wong, B. Wang, Z. Wang, K.F. Lao, A. Rosa, F. Wan, Spatial filtering in SSVEP-based BCIs: Unified framework and new improvements, *IEEE Trans. Biomed. Eng.* 67 (11) (2020) 3057–3072.
- [64] C.M. Wong, F. Wan, B. Wang, Z. Wang, W. Nan, K.F. Lao, P.U. Mak, M.I. Vai, A. Rosa, Learning across multi-stimulus enhances target recognition methods in SSVEP-based BCIs, *J. Neural Eng.* 17 (1) (2020) 016026.
- [65] Y. Zhang, G. Zhou, Q. Zhao, J. Jin, X. Wang, A. Cichocki, Spatial-temporal discriminant analysis for ERP-based brain-computer interface, *IEEE Trans. Neural Syst. Rehabil. Eng.* 21 (2) (2013) 233–243.
- [66] X. Liao, D. Yao, D. Wu, C. Li, Combining spatial filters for the classification of single-trial EEG in a finger movement task, *IEEE Trans. Biomed. Eng.* 54 (5) (2007) 821–831.
- [67] Y. Xie, K. Wang, J. Meng, J. Yue, L. Meng, W. Yi, T.-P. Jung, M. Xu, D. Ming, Cross-dataset transfer learning for motor imagery signal classification via multi-task learning and pre-training, *J. Neural Eng.* (2023).
- [68] R.T. Schirrmester, J.T. Springenberg, L.D.J. Fiederer, M. Glasstetter, K. Eggenberger, M. Tangermann, F. Hutter, W. Burgard, T. Ball, Deep learning with convolutional neural networks for EEG decoding and visualization, *Hum. Brain Mapp.* 38 (11) (2017) 5391–5420.
- [69] V.J. Lawhern, A.J. Solon, N.R. Waytowich, S.M. Gordon, C.P. Hung, B.J. Lance, EEGNet: a compact convolutional neural network for EEG-based brain-computer interfaces, *J. Neural Eng.* 15 (5) (2018) 056013.
- [70] O.B. Guney, M. Oblokulov, H. Ozkan, A deep neural network for SSVEP-based brain-computer interfaces, *IEEE Trans. Biomed. Eng.* 69 (2) (2022) 932–944.
- [71] Y. Li, J. Xiang, T. Kesavadas, Convolutional correlation analysis for enhancing the performance of SSVEP-based brain-computer interface, *IEEE Trans. Neural Syst. Rehabil. Eng.* 28 (12) (2020) 2681–2690.
- [72] A. Barachant, S. Bonnet, M. Congedo, C. Jutten, Riemannian geometry applied to BCI classification, in: *International Conference on Latent Variable Analysis and Signal Separation*, Springer, 2010, pp. 629–636.
- [73] A. Barachant, S. Bonnet, M. Congedo, C. Jutten, Multiclass brain-computer interface classification by Riemannian geometry, *IEEE Trans. Biomed. Eng.* 59 (4) (2011) 920–928.
- [74] P. Zanini, M. Congedo, C. Jutten, S. Said, Y. Berthoumieu, Transfer learning: A Riemannian geometry framework with applications to brain-computer interfaces, *IEEE Trans. Biomed. Eng.* 65 (5) (2018) 1107–1116.
- [75] L. Xu, M. Xu, Y. Ke, X. An, S. Liu, D. Ming, Cross-dataset variability problem in EEG decoding with deep learning, *Front. Hum. Neurosci.* 14 (2020).
- [76] P. Yuan, X. Chen, Y. Wang, X. Gao, S. Gao, Enhancing performances of SSVEP-based brain-computer interfaces via exploiting inter-subject information, *J. Neural Eng.* 12 (4) (2015) 046006.
- [77] P.L.C. Rodrigues, C. Jutten, M. Congedo, Riemannian procrustes analysis: Transfer learning for brain-computer interfaces, *IEEE Trans. Biomed. Eng.* 66 (8) (2019) 2390–2401.
- [78] K.J. Chiang, C.S. Wei, M. Nakanishi, T.P. Jung, Boosting template-based SSVEP decoding by cross-domain transfer learning, *J. Neural Eng.* 18 (1) (2021).
- [79] R. Bian, H. Wu, B. Liu, D. Wu, Small data least-squares transformation (sd-LST) for fast calibration of SSVEP-based BCIs, *IEEE Trans. Neural Syst. Rehabil. Eng.* 31 (2023) 446–455.
- [80] W. Zhang, D. Wu, Manifold embedded knowledge transfer for brain-computer interfaces, *IEEE Trans. Neural Syst. Rehabil. Eng.* 28 (5) (2020) 1117–1127.
- [81] A. Barredo Arrieta, N. Díaz-Rodríguez, J. Del Ser, A. Bennetot, S. Tabik, A. Barbado, S. Garcia, S. Gil-Lopez, D. Molina, R. Benjamins, R. Chatila, F. Herrera, Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI, *Inf. Fusion* 58 (2020) 82–115.